

Full Stack Development with MERN

Project Documentation

Date: 17 February 2026

1. Introduction

Project Title: HouseHunt – Finding Your Perfect Rental Home

Team ID : LTVIP2026TMIDS81826

Team Members:

Team Leader : Katta Aakash

Team member : Koruprolu Satya Divya

Team member : Sai Kammila

Team member : Sai Pradeep Bolla

2. Project Overview

Purpose:

HouseHunt is designed to provide a secure, scalable, and user-friendly rental property management platform. The system ensures verified owner onboarding, transparent booking workflows, and structured administrative governance. The platform eliminates traditional inefficiencies in rental systems by digitizing the end-to-end lifecycle from property listing to booking confirmation.

Features:

The application includes secure user registration and login, role-based authentication for Admin, Owner, and Renter, property CRUD operations for owners, structured booking request and approval workflow, and admin-controlled owner verification system. Each module is implemented following REST architectural principles.

3. Architecture

Frontend:

The frontend is developed using React.js with functional components and hooks. React Router is used for client-side routing, and Axios is used for secure API communication. State

management is handled using React's built-in hooks, ensuring modular and maintainable UI design.

Backend:

The backend is built using Node.js and Express.js. RESTful APIs manage authentication, property management, booking lifecycle, and admin approval mechanisms. Middleware is used for JWT verification and role-based authorization.

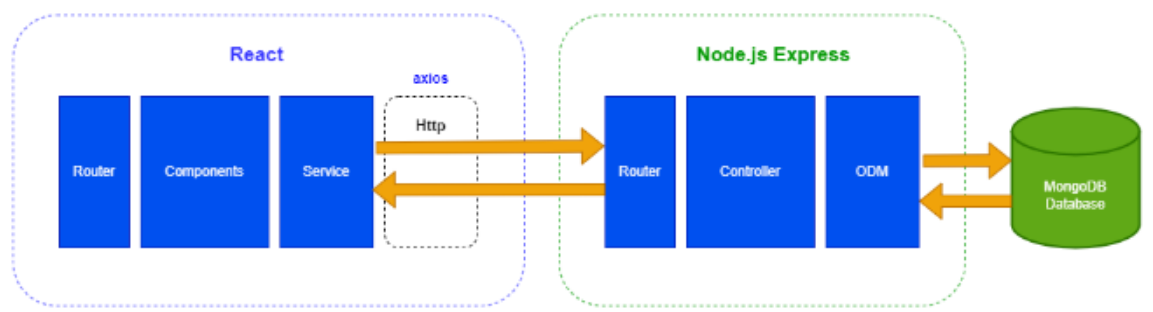
Database:

MongoDB serves as the NoSQL database. Mongoose ORM is used for schema modeling and validation. The system uses three primary collections: Users, Properties, and Bookings.

Database Schema Overview:

Collection	Key Fields	Description
Users	userId, email, password, role	Stores renter, owner, and admin accounts
Properties	propertyId, ownerId, rent, location	Stores property listings
Bookings	bookingId, userId, propertyId, status	Stores booking requests and status

System Architecture Diagram:



4. Setup Instructions

Prerequisites:

- Node.js
- MongoDB
- npm

Installation:

Step 1: Clone the repository

git clone <https://github.com/pradeep4129d/HouseHunt>

Step 2: Backend Setup

Create an env file in the backend root folder and add MongoDB connection URL

cd server

npm install

npm start

Step 3: Frontend Setup

cd client

npm install

npm start

5. Folder Structure

frontend Directory: Contains reusable components, pages, services for API calls, and main App.js entry point. Follows modular component-based architecture.

Ex:

frontend/src/components

frontend/src/context

frontend/src/styles

backend Directory: Organized into models (database schemas), routes (API endpoints), controllers (business logic), middleware (authentication & authorization), and server.js (application entry point).

Ex:

Backend/models

Backend/routes

Backend/server.js

6. Running the Application

To run the backend, navigate to the server directory and execute npm start. To run the frontend, navigate to the client directory and execute npm start. The frontend communicates with backend APIs running on configured port.

7. API Documentation

The backend exposes REST APIs for user management, property management, and booking lifecycle management. Each endpoint follows standard HTTP methods such as GET, POST, PUT, and DELETE.

API Endpoint Overview:

Method	Endpoint	Description
POST	/api/register	Register new user
POST	/api/login	Authenticate user
GET	/api/properties	Fetch properties
POST	/api/property	Add new property
POST	/api/booking	Create booking request

8. Authentication

Authentication is implemented using JSON Web Tokens (JWT). Upon successful login, a signed token is generated and stored on the client side. Middleware validates tokens for protected routes. Passwords are hashed using bcrypt before storage to ensure data security.

9. User Interface

The user interface is built using React.js and Bootstrap to ensure responsive design. Separate dashboards are provided for Renters, Owners, and Admin. The UI emphasizes usability, clarity, and accessibility.

- Registration Page
- Login Page
- Renter Dashboard
- Owner Dashboard
- Admin Dashboard
- Property Listing Page
- Booking Management Page

10. Testing

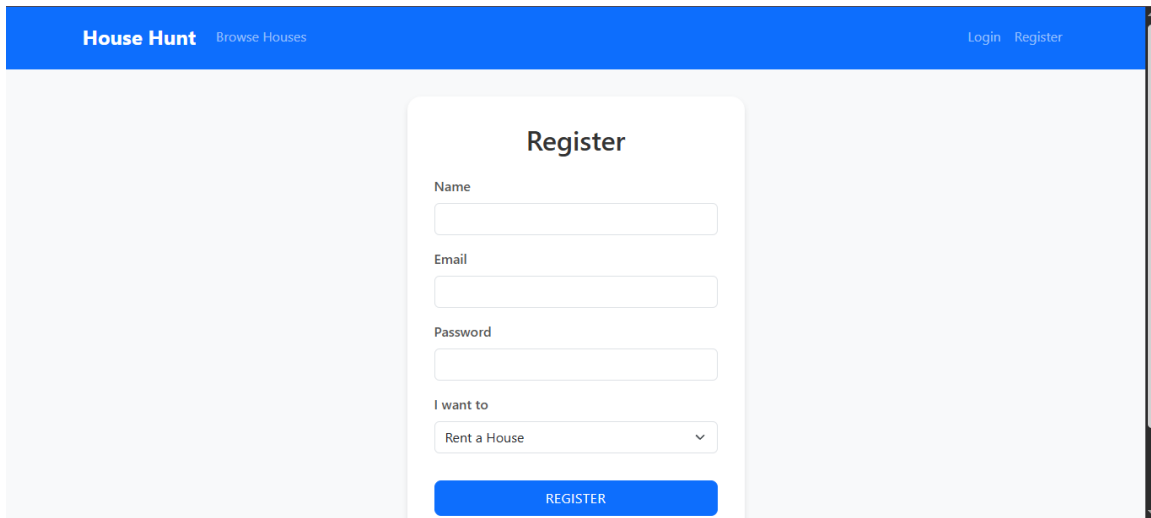
Comprehensive testing was conducted including API testing using Postman, functional testing for booking workflow, and role-based access validation. Edge cases and invalid inputs were tested to ensure robustness.

11. Screenshots or Demo

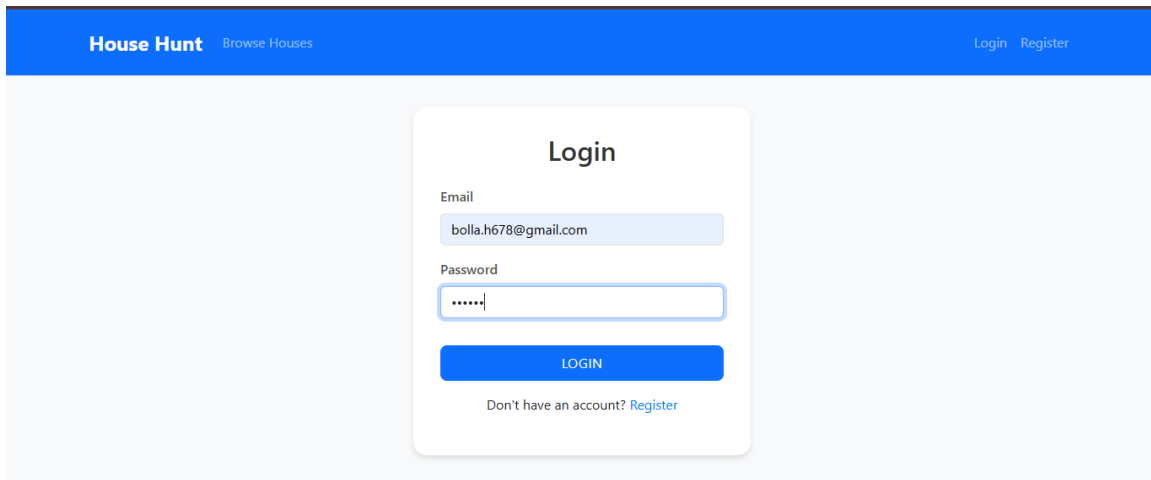
GitHub Repository: <https://github.com/pradeep4129d/HouseHunt>

Demo Link:

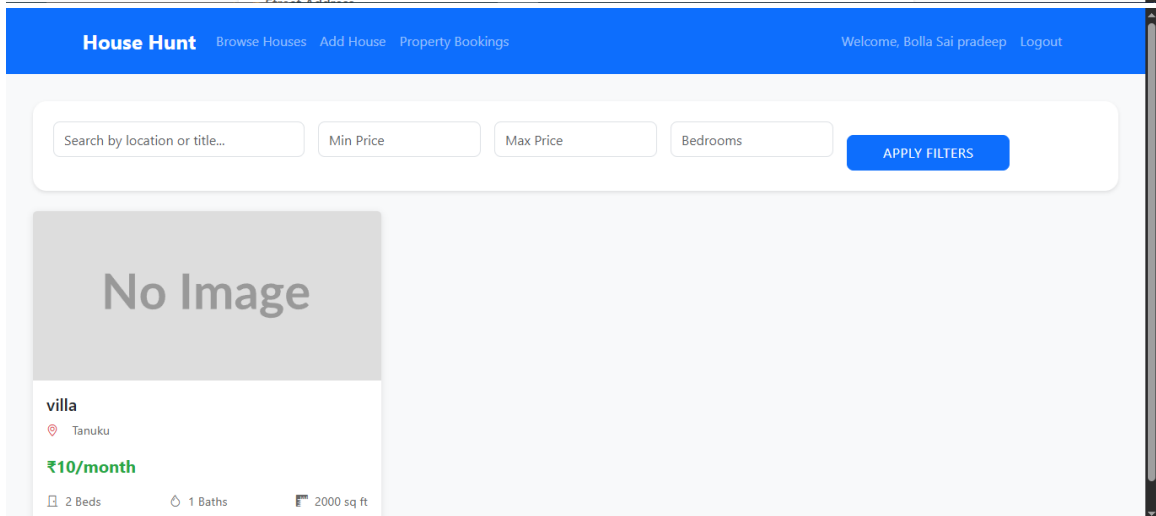
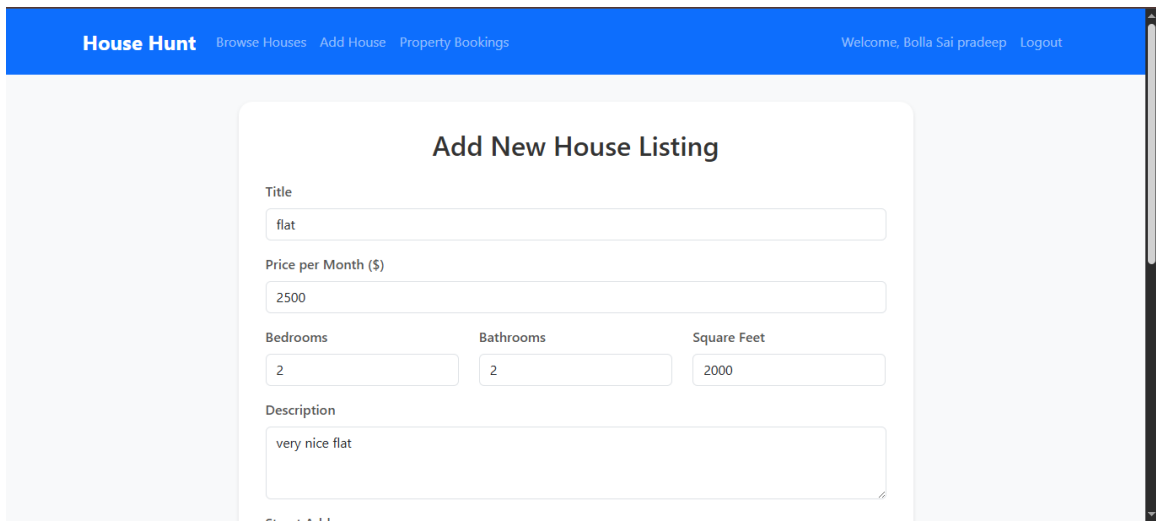
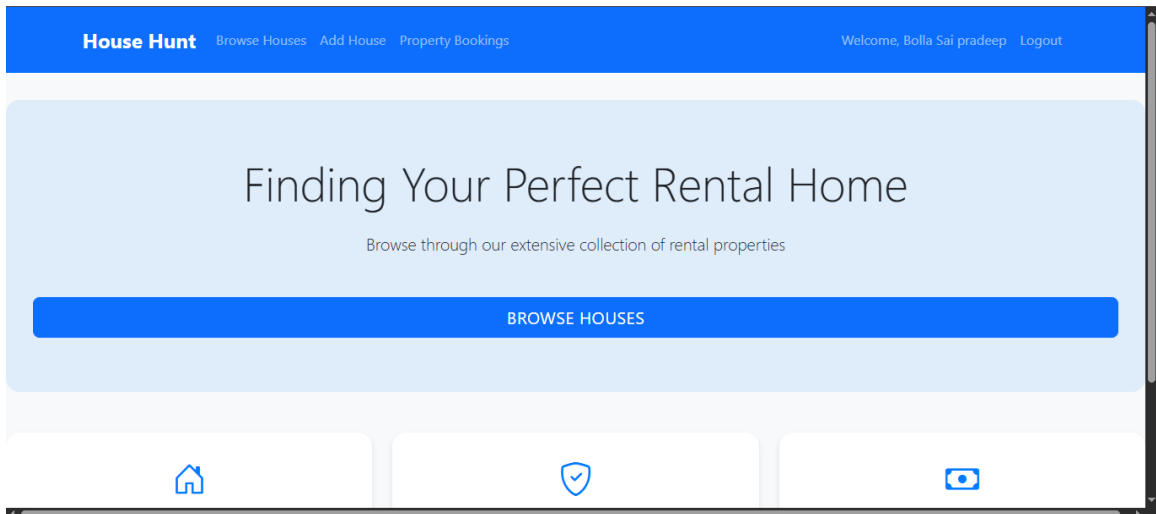
<https://drive.google.com/file/d/144QLKHteG6EORhQgW5xNwePHra8D4Vbi/view?usp=sharing>

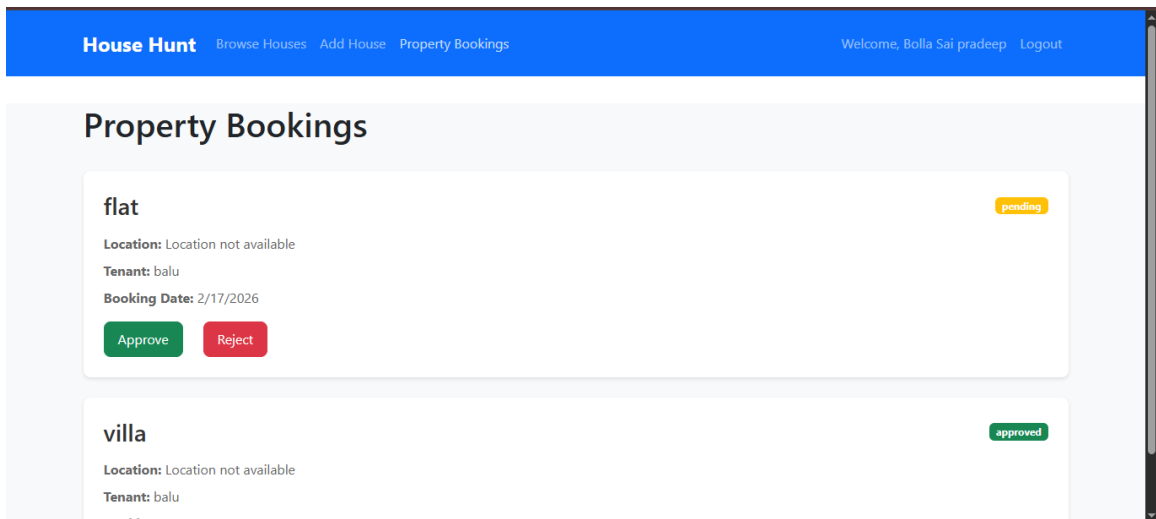
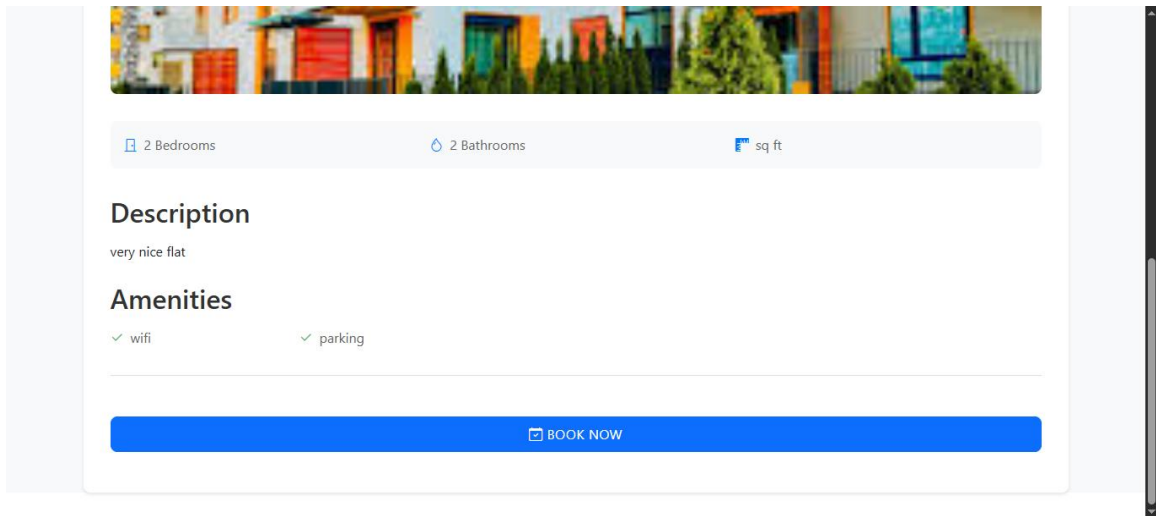


The screenshot shows the 'Register' page of the 'House Hunt' application. The page has a blue header with the text 'House Hunt' and a link 'Browse Houses'. On the right side of the header are links for 'Login' and 'Register'. The main content area is light gray and contains a white card with the title 'Register'. Inside the card, there are four input fields: 'Name', 'Email', 'Password', and a dropdown menu labeled 'I want to' with the option 'Rent a House' selected. At the bottom of the card is a blue button labeled 'REGISTER'.



The screenshot shows the 'Login' page of the 'House Hunt' application. The page has a blue header with the text 'House Hunt' and a link 'Browse Houses'. On the right side of the header are links for 'Login' and 'Register'. The main content area is light gray and contains a white card with the title 'Login'. Inside the card, there are two input fields: 'Email' with the value 'bolla.h678@gmail.com' and 'Password' with masked characters '.....'. At the bottom of the card is a blue button labeled 'LOGIN'. Below the card, there is a link that says 'Don't have an account? Register'.





12. Known Issues

The system currently does not integrate online payment gateways or real-time chat modules. Email verification and advanced notification systems are not implemented.

13. Future Enhancements

Future enhancements include payment gateway integration, AI-based recommendation engine, real-time messaging using WebSockets, containerized deployment using Docker, and cloud deployment on AWS or similar infrastructure.